

Non-Approved Algorithm Usage — pqc_crypto_module v0.1.0

Disclaimer: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

1. Non-Approved Algorithms

The following algorithms are present in the module for backward compatibility with pre-PQC ledger data. They are **outside the approved cryptographic boundary** and are not available when the module operates in Approved mode.

Algorithm	Crate	Purpose	FIPS Status
Ed25519	ed25519-dalek 2.1	Legacy digital signatures (pre-PQC blocks, wallets)	Not approved
SHA-256	sha2 0.10	Legacy block hashing	Not approved for this module
HMAC-SHA256	hmac 0.12 + sha2 0.10	Legacy MAC operations	Not approved for this module

Note: SHA-256 and HMAC-SHA256 are NIST-approved algorithms in general, but they are not part of this module's approved algorithm set. All approved hashing uses SHA3-256 (FIPS 202).

2. Legacy Functions

All non-approved functions are in `src/legacy.rs`:

Function	Algorithm	Signature
<code>legacy_ed25519_sign(sk, msg)</code>	Ed25519	<code>(&SigningKey, &[u8]) -> Result<Vec<u8>, CryptoError></code>
<code>legacy_ed25519_verify(pk, msg, sig)</code>	Ed25519	<code>(&[u8; 32], &[u8], &[u8]) -></code>

Function	Algorithm	Signature
legacy_sha256(data)	SHA-256	Result<(), CryptoError> (&[u8]) -> Result<[u8; 32], CryptoError>
legacy_hmac_sha256(key, data)	HMAC-SHA256	(&[u8], &[u8]) -> Result<Vec<u8>, CryptoError>

3. Gating Mechanisms

Non-approved algorithms are controlled by two independent mechanisms:

3.1 Runtime Guard (ensure_not_approved())

Every legacy function calls ensure_not_approved() as its first operation:

```
pub fn ensure_not_approved() -> Result<(), CryptoError> {
    match approved_mode::state() {
        ModuleState::Approved =>
            Err(CryptoError::NonApprovedAlgorithm),
        _ => Ok(()),
    }
}
```

Behavior by module state:

Module State	ensure_not_approved()	Legacy functions
Uninitialized	Ok(())	Available (for pre-migration use)
SelfTesting	Ok(())	Available (transient state)
Approved	Err(NonApprovedAlgorithm)	Blocked
Error	Ok(())	Available (but module is degraded)

When the module is in Approved state, any call to a legacy function returns:

```
Err(CryptoError::NonApprovedAlgorithm)
```

No cryptographic computation is performed.

3.2 Compile-Time Exclusion (approved-only feature flag)

The legacy.rs module begins with:

```
#[cfg(feature = "approved-only")]
compile_error!(
```

```
"Legacy crypto module is disabled in approved-only mode. \
  Remove usage of pqc_crypto_module::legacy::* or disable the
  approved-only feature."
);
```

When the crate is compiled with `--features approved-only`:

- The legacy module emits a compile error.
- Any code that imports or calls `pqc_crypto_module::legacy::*` fails to compile.
- This provides a hard guarantee that no non-approved algorithm is reachable in the binary.

Usage:

```
# Build with legacy algorithms available (default)
```

```
cargo build
```

```
# Build with legacy algorithms excluded at compile time
```

```
cargo build --features approved-only
```

4. Type Re-exports

The legacy module also re-exports type-level definitions from the underlying crates for struct compatibility:

Sub-module	Re-exports
<code>legacy::ed25519</code>	<code>SigningKey</code> , <code>VerifyingKey</code> , <code>Signature</code> , <code>Signer</code> , <code>Verifier</code> , <code>SignatureError</code>
<code>legacy::sha256</code>	<code>Sha256</code> , <code>Digest</code>
<code>legacy::hmac</code>	<code>Hmac</code> , <code>Mac</code> , <code>HmacSha256</code>
<code>legacy::rng</code>	<code>OsRng</code> , <code>Rng</code> , <code>RngCore</code> , <code>SliceRandom</code> , <code>rand_core</code>
<code>legacy::mldsa_raw</code>	<code>mldsa65</code> , <code>DetachedSignature</code> , <code>PublicKey</code> , <code>SecretKey</code>

These re-exports provide type definitions needed for pattern matching and struct fields in legacy code paths. They do **not** bypass the runtime guard — actual cryptographic operations must go through the guarded functions.

5. No Fallback Guarantee

The module is designed with no implicit fallback from approved to non-approved algorithms:

1. If an ML-DSA operation fails (e.g., invalid key), the error is returned directly. There is no fallback to Ed25519.
2. If SHA3-256 is available, SHA-256 is not silently used as an alternative.

3. The approved API (`pqc_crypto_module::api`) does not expose any Ed25519, SHA-256, or HMAC-SHA256 functions.

This is verified by: - `tests/no_fallback.rs` — confirms no classical fallback on ML-DSA failure. - `tests/approved_vs_legacy.rs` — confirms legacy is blocked in Approved mode and no implicit fallback occurs.

6. Migration Path

The intended migration path for Cerulean Ledger:

1. **Phase 1 (current):** Both approved and legacy algorithms available. Legacy used only for verifying pre-existing blocks.
2. **Phase 2:** All new operations use approved algorithms. Legacy used only for historical verification.
3. **Phase 3:** Build with `--features approved-only` to eliminate legacy code from the binary entirely.

At Phase 3, the only algorithms in the compiled binary are ML-DSA-65, ML-KEM-768, and SHA3-256.